SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

FACULTY OF COMPUTING

BACHELOR OF SCIENCE (HONOURS) IN INFORMATION TECHNOLOGY

MULTI-STREAM FUSION MODEL FOR REAL-TIME SINHALA SIGN LANGUAGE RECOGNITION USING HAND GESTURES, FACIAL EXPRESSIONS, AND BODY POSTURE

IT22304674 - Liyanage M.L.I.S.

DECLARATION

I hereby declare that the work presented in this thesis is my own and that it has not been submitted for any other degree, diploma, or qualification at this or any other institution.

All sources of information used in this thesis have been duly acknowledged and referenced. The ideas and views expressed in this thesis are entirely my own unless explicitly attributed to another source.

I understand that any form of plagiarism or misrepresentation is a serious academic offence and I confirm that this thesis is free from such practices.

Student Name : Liyanage M.L.I.S.
Student ID : IT22304674
Programme : BSc (Hons) in Information Technology
Faculty : Computing
Institution : Sri Lanka Institute of Information Technology (SLIIT)
Date : April 2026

Signature of Student : _____

Certified by:

Supervisor Name : _____

Signature : _____

Date : _____

ABSTRACT

Sign language recognition systems that rely exclusively on hand gesture analysis fail to capture the full communicative intent of a signer because sign languages encode meaning simultaneously through hand shapes, facial expressions, and body posture. This research presents a real-time Sinhala Sign Language (SSL) recognition system — referred to as the SSL Reader — that addresses this limitation through a purpose-built Multi-Stream Fusion Model that analyses all three modalities concurrently.

The system employs Google MediaPipe Tasks API (v0.10.21) to extract a 457-dimensional feature vector from each video frame, combining 126 dimensions of hand landmark coordinates (both hands, 21 landmarks each), 232 dimensions of facial features (60 key landmarks plus 52 expression blendshape scores), and 99 dimensions of full body pose landmarks (33 joints). Each modality is processed by a dedicated sub-network whose architecture is matched to the temporal characteristics of that modality: Temporal Convolutional Networks (TCNs) with exponentially dilated convolutions handle the rapid dynamics of hand gestures and the continuous changes in facial expression; a Bidirectional Long Short-Term Memory (BiLSTM) network handles the slower, context-dependent changes in body posture. An attention-based fusion module learns to dynamically weight the three streams for each individual sign before a two-layer classifier produces the final prediction.

Four model architectures were systematically developed and evaluated: Multimodal BiLSTM (~47% validation accuracy), Multimodal Transformer (~41%), Hybrid LSTM-Transformer (~52%), and the final Multi-Stream Fusion Model (76.3% validation accuracy, 80.1% test accuracy on 383 Sinhala sign classes). The 30-percentage-point improvement over the best single-stream architecture demonstrates that modality-specific processing before fusion is the critical design decision.

The model was trained on 8,472 videos spanning 383 Sinhala sign vocabulary items under severe data scarcity (~22 videos per class), addressed through skeleton-level augmentation (seven stochastic operations), label smoothing, and dropout regularisation. The trained model is deployed as a Flask REST API that streams predictions to a React Native mobile application, forming the recognition module of the broader Intelligent Sinhala Sign Language Communication Platform.

Keywords: Sinhala Sign Language, Multi-Stream Fusion, Temporal Convolutional Network, BiLSTM, MediaPipe, Attention Fusion, Multi-Modal Recognition, Real-Time Inference

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all who supported me throughout the development of this research project.

First and foremost, I thank my academic supervisor for providing continuous guidance, constructive feedback, and encouragement throughout the research process. Their expertise in deep learning and computer vision was invaluable in shaping the direction of this work.

I extend my appreciation to the Intelligent Sinhala Sign Language Communication Platform project team — the developers of the Sound Alert, Adaptive Learning, and Two-Way Communication components — for their collaborative effort in building a cohesive platform. The integration work across components was made possible through their cooperation and technical expertise.

I am grateful to the deaf and hearing-impaired community members in Sri Lanka who participated in data collection and provided feedback on the system. Their involvement is the purpose of this work, and their perspectives were essential in shaping a system that is genuinely useful.

I thank the Sri Lanka Institute of Information Technology (SLIIT) for providing the computational resources, laboratory access, and academic environment that made this research possible.

My sincere thanks go to my family for their patience, moral support, and understanding throughout the demanding period of thesis preparation.

Finally, I acknowledge the open-source communities behind PyTorch, Google MediaPipe, and the scientific community whose published research formed the foundation of this work.

TABLE OF CONTENTS

DECLARATION ii
ABSTRACT iii
ACKNOWLEDGEMENT iv
TABLE OF CONTENTS v
LIST OF TABLES vi
LIST OF FIGURES vii
LIST OF ABBREVIATIONS viii

CHAPTER 1: INTRODUCTION 1
1.1 Background Literature 2
1.1.1 Sign Language Recognition: An Overview 2
1.1.2 Traditional Machine Learning Approaches 3
1.1.3 Deep Learning in Sign Language Recognition 4
1.1.4 Multi-Modal Sign Language Recognition 5
1.1.5 MediaPipe for Pose Estimation 7
1.1.6 Sign Language Recognition in Sri Lanka 8
1.2 Research Gap 9
1.3 Research Problem 11
1.4 Research Objectives 12

CHAPTER 2: METHODOLOGY 14
2.1 System Overview 14
2.2 Dataset 15
2.3 Feature Extraction 18
2.4 Root-Relative Coordinate Normalisation 23
2.5 Data Augmentation 25
2.6 Model Architecture Development 27
2.7 Final Model — Multi-Stream Fusion Model 30
2.8 Training Configuration 35
2.9 Commercialisation Aspects 37
2.10 Testing and Implementation 40

CHAPTER 3: RESULTS AND DISCUSSION 45
3.1 Results 45
3.2 Research Findings 47
3.3 Discussion 49
3.4 Summary of Student Contribution 52

CHAPTER 4: CONCLUSION 54

REFERENCES 57
GLOSSARY 60
APPENDICES 62

LIST OF TABLES

Table 1.1 Summary of related sign language recognition systems 9
Table 2.1 signVideo dataset statistics 16
Table 2.2 Dataset split distribution 17
Table 2.3 Feature stream dimensions 23
Table 2.4 Data augmentation operations 26
Table 2.5 Architecture comparison of all four models 29
Table 2.6 Multi-Stream Fusion Model architecture summary 34
Table 2.7 Training hyperparameters 36
Table 2.8 Commercialisation aspects summary 39
Table 2.9 Test case results summary 44
Table 3.1 Final model performance metrics 46
Table 3.2 Validation accuracy comparison across architectures 47

LIST OF FIGURES

Figure 1.1 Scope of modalities in sign language communication 5
Figure 2.1 System architecture overview 14
Figure 2.2 Feature extraction pipeline 19
Figure 2.3 Hand landmark positions (21 points per hand) 20
Figure 2.4 Selected facial landmark groups (60 landmarks) 21
Figure 2.5 Body pose landmark positions (33 points) 22
Figure 2.6 Root-relative normalisation for each stream 24
Figure 2.7 Augmentation operations visualised on hand landmarks 26
Figure 2.8 Model architecture evolution 28
Figure 2.9 Multi-Stream Fusion Model architecture diagram 31
Figure 2.10 TCN block structure with residual connection 32
Figure 2.11 Attention fusion mechanism diagram 34
Figure 2.12 Training loss and validation accuracy curves 37
Figure 2.13 Commercialisation deployment model 38
Figure 3.1 Validation accuracy across training epochs 46
Figure 3.2 Architecture accuracy comparison bar chart 48
Figure 3.3 Attention weight distributions for sample signs 51

LIST OF ABBREVIATIONS

API Application Programming Interface
BiLSTM Bidirectional Long Short-Term Memory
BN Batch Normalisation
CNN Convolutional Neural Network
CORS Cross-Origin Resource Sharing
CPU Central Processing Unit
CTC Connectionist Temporal Classification
CUDA Compute Unified Device Architecture
FC Fully Connected
FPS Frames Per Second
GPU Graphics Processing Unit
HOG Histogram of Oriented Gradients
HTTP Hypertext Transfer Protocol
HMM Hidden Markov Model
IoT Internet of Things
JSON JavaScript Object Notation
LSTM Long Short-Term Memory
ML Machine Learning
MVP Minimum Viable Product
POST HTTP POST Method
ReLU Rectified Linear Unit
REST Representational State Transfer
RGB Red Green Blue
RNN Recurrent Neural Network
SaaS Software as a Service
SLR Sign Language Recognition
SSL Sinhala Sign Language
SVM Support Vector Machine
TCN Temporal Convolutional Network
UI User Interface
UNESCO United Nations Educational, Scientific and Cultural Organization
USD United States Dollar
val Validation

CHAPTER 1: INTRODUCTION

Sign language is the primary and natural language of millions of deaf and hearing-impaired people around the world. Unlike spoken languages, which rely on the auditory-vocal channel, sign languages are visual-spatial languages that transmit meaning through coordinated hand shapes, hand movements, facial expressions, and body posture. Sign languages are full, natural languages with their own grammar, syntax, and vocabulary — they are not simplified codes or derivatives of spoken languages.

In Sri Lanka, the deaf and hearing-impaired community comprises an estimated 200,000 individuals who communicate primarily through Sinhala Sign Language (SSL). Despite this substantial population, SSL remains largely unknown to the general hearing public, creating a pervasive communication barrier that affects nearly every aspect of daily life. A deaf individual visiting a hospital, a school, a government office, or a shop confronts the same barrier: there is no shared language between them and the hearing person they need to communicate with. Professional sign language interpreters exist but are scarce and expensive, and are unavailable for the spontaneous, informal interactions of everyday life.

Automated sign language recognition — using computer vision and machine learning to interpret sign language from video input — offers a technological approach to breaking down this barrier. If a smartphone or computer can recognise what a person is signing in real time and produce the corresponding text, it becomes possible for deaf individuals to communicate independently with hearing people who have access to that device.

This thesis presents the design, implementation, and evaluation of the SSL Reader component: a deep learning system that performs real-time Sinhala Sign Language recognition using a novel multi-modal approach. While the majority of published sign language recognition systems analyse only hand gestures, the SSL Reader simultaneously analyses hand gestures, facial expressions, and body posture. This three-modality approach mirrors how human interpreters read sign language and produces significantly more accurate recognition than hand-only systems.

The SSL Reader is developed as one component of a larger platform — the Intelligent Sinhala Sign Language Communication Platform (ISSLCP) — that also includes text-to-sign animation, environmental sound alerting, and adaptive learning modules. Together, these components aim to address communication accessibility and safety needs of the deaf community in Sri Lanka.

This chapter reviews the background literature, identifies the research gap, states the research problem, and defines the research objectives. Subsequent chapters describe the methodology (Chapter 2), present and discuss the results (Chapter 3), and draw conclusions (Chapter 4).

1.1 Background Literature

1.1.1 Sign Language Recognition: An Overview

The field of automatic sign language recognition (SLR) has been studied for over three decades, beginning with early systems in the 1990s that recognised small vocabularies of static hand postures (Liddell and Johnson, 1989). The fundamental challenge in SLR is significantly greater than in many other pattern recognition domains: signs are not static postures but dynamic temporal sequences, signers vary widely in their signing style, speed, and body proportions, and the visual appearance of a sign depends heavily on lighting, background, and camera angle.

Sign languages can be analysed at several levels. At the phonological level, signs are composed of simultaneous parameters including handshape, location, movement, orientation, and non-manual features (facial expressions, head tilt, mouth movements, and body posture). This simultaneous multi-channel structure is what makes sign language recognition fundamentally different from handwriting recognition or gesture spotting in isolation.

The problem is typically framed as: given a video sequence of a person signing, classify the sequence as one of N known signs. Three main sub-tasks are recognised in the literature: (a) isolated sign recognition — classify individual, segmented signs; (b) sign spotting — detect sign boundaries within continuous video; and (c) continuous sign language recognition — recognise full sentences. This research addresses isolated sign recognition, which is a necessary precondition for the more challenging continuous recognition task.

1.1.2 Traditional Machine Learning Approaches

Early SLR systems relied on hand-engineered features extracted from video frames, fed into classical machine learning classifiers. Common feature engineering approaches included skin colour segmentation, Histogram of Oriented Gradients (HOG) descriptors, optical flow fields, Discrete Cosine Transform (DCT) coefficients, and depth map analysis from RGB-D cameras.

These features were fed into classifiers including Support Vector Machines (SVMs), Hidden Markov Models (HMMs), and k-Nearest Neighbours classifiers. HMM-based systems in particular achieved reasonable performance on constrained laboratory datasets but suffered from several limitations: they required large amounts of manually annotated data, hand-engineered features required careful domain expertise, and models were brittle — performing poorly when the test signer, lighting, or background differed from training conditions.

1.1.3 Deep Learning in Sign Language Recognition

The introduction of deep convolutional neural networks (CNNs) and recurrent neural networks (RNNs) transformed sign language recognition. The key advantage of deep learning approaches is end-to-end learning: rather than requiring manually engineered features, the network learns to extract relevant representations directly from raw pixel or landmark data during training.

CNNs trained on single-frame images achieved strong performance on fingerspelling recognition (Pugeault and Bowden, 2011). However, single-frame CNNs cannot model the temporal dynamics of dynamic signs. Long Short-Term Memory (LSTM) networks proved well-suited to sign language recognition because they can model sequences of varying length and capture long-range temporal

dependencies (Huang et al., 2018). Transformer architectures (Vaswani et al., 2017) demonstrated that attention-based global context modelling was advantageous for longer signing sequences, though they are data-hungry. Temporal Convolutional Networks (TCNs), proposed by Bai et al. (2018), offer parallelised training with stable gradients and explicit receptive field control, making them competitive with LSTMs for sequence modelling.

1.1.4 Multi-Modal Sign Language Recognition

The recognition that hand gestures alone are insufficient for complete sign language understanding has motivated a growing body of research on multi-modal approaches. Sign languages are multi-channel communication systems in which different parts of the body simultaneously carry different types of information.

Hands serve as the primary articulators. Facial expressions perform grammatical functions: raised eyebrows mark yes/no questions, furrowed eyebrows mark wh-questions, and head shakes with specific expression patterns mark negation. Body posture communicates subject/object referencing, emphasis, and role shift. The importance of non-manual features has long been recognised in sign language linguistics (Sandler and Lillo-Martin, 2006).

Koller et al. (2015) demonstrated that including mouth shape features alongside hand features improved recognition accuracy for German Sign Language. Jiang et al. (2021) proposed a skeleton-aware multi-stream network for Chinese Sign Language using separate streams for hands and body, confirming that modality-specific processing produced better recognition than single-stream baselines. However, no published system has applied a full three-stream approach (hands + face + body) with modality-dedicated architectures and learned attention fusion.

1.1.5 MediaPipe for Pose Estimation

The feature extraction backbone for the SSL Reader is Google MediaPipe, an open-source framework for building on-device perception pipelines (Lugaresi et al., 2019). MediaPipe provides three task models relevant to this research: Hand Landmarker (21 three-dimensional landmark coordinates per hand), Face Landmarker (468 three-dimensional landmarks plus 52 blendshape scores representing facial muscle-group activation levels), and Pose Landmarker (33 body landmarks). MediaPipe has been adopted in multiple recent SLR studies precisely because it provides accurate, real-time, cross-platform landmark extraction without requiring depth sensors.

1.1.6 Sign Language Recognition in Sri Lanka

Published research on automated Sinhala Sign Language (SSL) recognition is sparse compared to the extensive literature on American Sign Language (ASL), German Sign Language (DGS), or Chinese Sign Language (CSL). Existing published SSL recognition systems address vocabularies of 10 to approximately 50 signs, rely exclusively on hand gesture features, often use traditional ML classifiers, and are typically trained on a single signer. Kumara and Gunasekara (2019) demonstrated SSL recognition on a 25-sign vocabulary but did not address scalability, temporal dynamics, or non-manual features.

The SSL Reader presented in this thesis represents a substantial advancement: a 383-class vocabulary (more than 7× larger than any published SSL system), three-modality feature extraction, deep learning with a novel multi-stream architecture, and real-time deployment on a mobile-accessible platform.

1.2 Research Gap

The review of the literature reveals five specific gaps that motivate this research:

Gap 1: Insufficient Modality Coverage. The overwhelming majority of published SLR systems — including all published SSL recognition systems — use only hand gesture features, ignoring facial expressions and body posture that are integral to complete sign language communication.

Gap 2: Single-Stream Processing. The small number of multi-modal SLR systems that exist typically concatenate features from different modalities at the input level and process them through a single shared model, preventing specialisation for each modality's different temporal dynamics.

Gap 3: Limited Expression Intensity Information. Existing systems that include face features use only facial landmark positions. The recent availability of face blendshape scores through MediaPipe provides richer expression information not previously exploited.

Gap 4: Lack of Large-Scale SSL Systems. No published SSL system addresses a vocabulary larger than 50 signs. The 383-class vocabulary in this work is 7× larger than any prior published SSL system.

Gap 5: Absence of Mobile-Accessible SSL Recognition. Prior SSL research has been confined to laboratory settings. No published SSL system is deployed in a form accessible to general users on mobile devices.

Table 1.1: Summary of related sign language recognition systems

System	Language	Vocab.	Modalities	Architecture
Starner & Pentland (1995)	ASL	40	Hand only	HMM
Huang et al. (2018)	CSL	500	Hand only	CNN-LSTM
Camgoz et al. (2020)	GSL	1,000+	Hand + Mouth	Transformer
Jiang et al. (2021)	CSL	1,000+	Hand + Body	Multi-Stream
Kumara & Gunasekara (2019)	SSL	25	Hand only	SVM
This work	SSL	383	Hand + Face + Body	Multi-Stream Fusion TCN+LSTM

1.3 Research Problem

The central research problem is:

> *"Existing Sinhala Sign Language recognition systems achieve limited accuracy and vocabulary coverage because they rely exclusively on hand gesture features, ignoring the facial expressions and body posture that are integral to complete sign language communication. Furthermore, systems that do combine multiple feature types process them through a single shared model that cannot specialise for the different temporal dynamics of each modality."*

This problem has four specific dimensions: (1) Modality Incompleteness — current SSL systems capture only one of the three channels through which SSL is expressed; (2) Architectural Mismatch — single-stream architectures cannot simultaneously optimise for hand gesture dynamics, facial expression dynamics, and body posture dynamics; (3) Data Scarcity — approximately 22 training videos per class creates strong overfitting risk; and (4) Deployment Accessibility — academic SLR research has largely remained in the laboratory rather than being deployed to general users.

1.4 Research Objectives

Primary Objective: To design, implement, and evaluate a real-time Sinhala Sign Language recognition system that achieves high accuracy on a large vocabulary by simultaneously analysing hand gestures, facial expressions, and body posture through a multi-stream neural architecture.

Four specific objectives operationalise this primary goal: (1) Multi-Modal Feature Extraction — design a pipeline capturing three modalities from video input using MediaPipe Tasks API; (2) Multi-Stream Architecture Design — develop a neural network with dedicated sub-networks for each modality and an attention-based fusion module; (3) Robust Training Under Data Scarcity — achieve reliable generalisation despite ~22 training samples per class through skeleton-level augmentation, label smoothing, dropout, and cosine annealing; and (4) Real-Time Deployment — deploy the trained model as a Flask REST API integrated with a React Native mobile application.

CHAPTER 2: METHODOLOGY

This chapter describes the complete methodology for the SSL Reader component, from data collection and feature extraction through to model architecture, training, commercialisation considerations, and system deployment.

2.1 System Overview

The SSL Reader is a machine learning-based video recognition system that accepts camera input and produces predicted Sinhala sign labels in real time. The system consists of four major stages:

Stage 1 — Feature Extraction: Each video frame is processed by three MediaPipe landmark detection models running in parallel (Hand Landmarker, Face Landmarker, Pose Landmarker). Their outputs are concatenated into a 457-dimensional feature vector per frame, accumulated over a 60-frame rolling buffer.

Stage 2 — Preprocessing: Extracted landmark coordinates are normalised using root-relative normalisation. During training, stochastic skeleton-level augmentation is also applied.

Stage 3 — Temporal Classification: The normalised 60 × 457 feature tensor is passed to the Multi-Stream Fusion Model, which processes hand, face, and pose features through dedicated sub-networks and fuses them with attention to produce a probability distribution over 383 sign classes.

Stage 4 — Output and Deployment: The predicted sign label (in Sinhala Unicode) and its confidence score are displayed on-screen or returned via the REST API to the mobile application.

2.2 Dataset

2.2.1 Dataset Source and Collection

The training and evaluation data is the signVideo dataset, a locally collected video corpus of Sinhala Sign Language. Videos were recorded by signers against a relatively uniform background. Each sign class is stored as a separate directory containing multiple MP4 video files. Videos vary in frame rate (typically 24–30 fps) and resolution but consistently depict a single signer. The vocabulary covers common conversational scenarios including greetings, relationships, objects, actions, time expressions, and grammatical function words.

2.2.2 Vocabulary and Statistics

Table 2.1: signVideo dataset statistics

Statistic	Value
Total sign classes	383
Total videos	8,472

Average videos per class	~22
Minimum videos per class	~15 (varies)
Video format	MP4
Vocabulary categories	Greetings, People, Nouns, Verbs, Numbers, Days, Months, Colours, Adjectives, Adverbs, Prepositions, Conjunctions, Determiners, Interjections, Places

2.2.3 Dataset Splits

The dataset was divided using stratified splitting to ensure every class is proportionally represented in each partition.

Table 2.2: Dataset split distribution

Split	Videos	Percentage
Training	5,827	68.8%
Validation	1,358	16.0%
Test	1,287	15.2%
Total	8,472	100.0%

2.2.4 Data Scarcity Challenge

With an average of approximately 22 videos per class, the dataset is severely under-resourced by the standards of modern deep learning. For comparison, ImageNet contains approximately 1,000 images per class, and large-scale sign language datasets such as WLASL contain 3–100 videos per sign. Every design decision in the architecture and training procedure was made with data scarcity in mind.

2.3 Feature Extraction

2.3.1 MediaPipe Framework

Feature extraction was performed using Google MediaPipe Tasks API version 0.10.21. Three pre-trained task models were deployed with GPU-accelerated float16 precision: Hand Landmarker, Face Landmarker (with blendshapes output enabled), and Pose Landmarker (full model variant). Each video was processed frame by frame using OpenCV. A fixed temporal window of exactly 60 frames was used: videos with fewer frames were zero-padded at the end; videos with more frames were uniformly subsampled to 60 frames.

2.3.2 Hand Stream

The Hand Landmarker detected and tracked both hands simultaneously, providing 21 three-dimensional landmark coordinates for each hand (wrist, thumb, index, middle, ring, and pinky finger joints). For both hands: $2 \times 21 \times 3 = 126$ dimensions total. Detection thresholds were set to 0.3 to

improve robustness for fast-moving and partially occluded hands. When a hand was not detected, the corresponding 63-dimensional block was filled with zeros.

2.3.3 Face Stream

MediaPipe's Face Landmarker provides 468 three-dimensional landmarks. Instead of using all 468, 60 key landmarks were selected based on their linguistic relevance to sign language: 12 eye landmarks (capturing openness, gaze, and squinting), 10 eyebrow landmarks (capturing brow raising and furrowing, which are grammatically obligatory for questions in many sign languages), 14 mouth landmarks, 4 nose landmarks, and 20 face oval landmarks.

Additionally, 52 MediaPipe face blendshape scores were included. Blendshapes are scalar values between 0 and 1 representing specific facial muscle group activations (e.g., jaw_open, brow_inner_up, eye_wide_left). Total face feature dimensions: 60 × 3 (landmarks) + 52 (blendshapes) = 232.

2.3.4 Pose Stream

The Pose Landmarker detects 33 body landmarks as (x, y, z) normalised coordinates: 33 × 3 = 99 dimensions. The 33 landmarks cover head, shoulders, elbows, wrists, hips, knees, ankles, heels, and foot points. All 33 landmarks are retained to preserve full-body posture context.

2.3.5 Combined Feature Vector

Table 2.3: Feature stream dimensions

Stream	Dimensions	Source
Hand stream	126	2 × 21 × 3
Face stream	232	60×3 + 52
Pose stream	99	33 × 3
Total	457	per frame

Each video was represented as a tensor of shape (60 frames × 457 features), which formed the input to all model architectures.

2.3.6 Feature Caching

To avoid repeating MediaPipe feature extraction on every training epoch, extracted features were cached to disk as Python pickle files. Feature caching reduced the effective data loading time during training to approximately the time required for simple file I/O, rather than the multiple seconds per video required for MediaPipe extraction.

2.4 Root-Relative Coordinate Normalisation

Raw MediaPipe coordinates are expressed in the image coordinate system, normalised to [0, 1] relative to image dimensions. Root-relative normalisation was applied to make features position-

invariant. For each stream, a designated root landmark is subtracted from all landmarks, translating the entire stream so the root is at the origin.

Hand Normalisation: The wrist landmark (index 0) was designated as the root for each hand independently. All 21 landmarks were translated by subtracting the wrist position, making hand features invariant to absolute position in the image frame. When a hand was not detected (all-zero block), normalisation left it unchanged.

Face Normalisation: The nose tip (index 36 within the filtered 60-landmark set) was designated as the root. All 60 key landmarks were translated by subtracting the nose tip position. The 52 blendshape scores, which are scalar values, were not modified.

Pose Normalisation: The mid-shoulder point — the average of the left shoulder (landmark 11) and right shoulder (landmark 12) — was designated as the root. All 33 pose landmarks were translated by subtracting this mid-shoulder point.

This normalisation was applied identically in both the training pipeline and the inference pipeline. Consistency between training-time and inference-time normalisation is critical: a mismatch would cause the model to receive differently distributed features at inference, degrading accuracy.

2.5 Data Augmentation

The severe data scarcity (~22 videos per class) required substantial artificial data augmentation performed directly on extracted skeleton landmarks — referred to as skeleton-level augmentation. This approach is computationally cheap and preserves the semantic validity of the landmark representation. Seven stochastic operations were implemented:

Table 2.4: Data augmentation operations

Operation	Parameters	Purpose
Spatial rotation	Angle ~ Uniform(-5°, +5°) on hand and pose streams	Simulates camera angle variation
Spatial scaling	Scale ~ Uniform(0.9, 1.1) multiplicatively	Simulates signer at different distances
Gaussian noise injection	$\sigma = 0.002$ on all coordinate features	Sensor noise robustness; prevents position memorisation
Temporal shift	±10% of frames, $p = 0.30$; vacated frames zero-padded	Simulates different signing onset timing
Time warping	Speed ~ Uniform(0.8, 1.2), $p = 0.25$; re-interpolated to 60 frames	Simulates faster or slower signers
Frame dropping	Up to 20% of frames zeroed, $p = 0.20$	Simulates brief detection failures
Time masking	Contiguous frame block zeroed, $p = 0.15$	Simulates temporal occlusion

Global application probability: 0.70 (30% of training samples pass through augmentation unchanged). Horizontal flipping was deliberately excluded because in sign language the distinction between left-handed and right-handed execution carries semantic meaning for some signs, and mirroring a sign could transform it into a different sign.

2.6 Model Architecture Development

Four architectures were designed and evaluated in sequence. Each iteration was motivated by analysis of the limitations observed in its predecessor.

2.6.1 Model 1 — Multimodal BiLSTM

Input: (batch, 60, 457). Architecture: Linear projection 457 → 256; 2-layer Bidirectional LSTM hidden size 256 per direction; soft attention over LSTM outputs; two-layer classifier 512 → 256 → 383. Parameters: ~3.1M. Validation accuracy: ~47%. Limitation: single-stream design concatenated all features at input; the LSTM was forced to apply the same filters to all three feature types simultaneously, showing strong bias toward the higher-dimensional face stream while under-weighting hand and pose streams.

2.6.2 Model 2 — Multimodal Transformer

Input: (batch, 60, 457). Architecture: Linear embedding 457 → 256; sinusoidal positional encoding; 4-layer Transformer encoder (d_model=256, 8 heads); global average pooling; two-layer classifier 256 → 383. Parameters: ~4.7M. Validation accuracy: ~41%. Limitation: Transformer performed worse than LSTM despite having more parameters, consistent with known data-hunger of global attention mechanisms. With only ~22 training samples per class, the Transformer could not learn reliable global attention patterns.

2.6.3 Model 3 — Hybrid LSTM-Transformer

Input: (batch, 60, 457). Architecture: Bidirectional LSTM hidden size 256 per direction (→ 512 output); Linear projection 512 → 256; sinusoidal positional encoding; 2-layer Transformer encoder; global average pooling; two-layer classifier 256 → 383. Parameters: ~5.2M. Validation accuracy: ~52%. Limitation: achieved the best single-stream accuracy but still processed all three modalities through a single shared pipeline, leaving the fundamental architectural mismatch problem unaddressed.

2.6.4 Model 4 — Multi-Stream Fusion Model (Final)

The key insight from models 1–3 was that no single-stream architecture could effectively specialise for three semantically and temporally heterogeneous input modalities simultaneously. The solution was architectural: route each modality through its own dedicated sub-network matched to its temporal characteristics, then combine the resulting representations. This led to the Multi-Stream Fusion Model described in Section 2.7, which achieved a 24-percentage-point improvement (52% → 76.3%) while simultaneously reducing the parameter count (5.2M → 2.52M).

Table 2.5: Architecture comparison of all four models

Model	Params	Val Acc	Key Characteristic
Model 1: Multimodal BiLSTM	~3.1M	~47%	Single-stream LSTM; conflates modalities
Model 2: Multimodal Transformer	~4.7M	~41%	Data-hungry; worse than LSTM at this scale
Model 3: Hybrid LSTM-Transformer	~5.2M	~52%	Single-stream hybrid; cannot specialise
Model 4: Multi-Stream Fusion (FINAL)	~2.52M	76.3%	Dedicated streams with attention fusion

2.7 Final Model — Multi-Stream Fusion Model

2.7.1 Design Rationale

The three modalities have fundamentally different temporal dynamics. Hand gestures are characterised by rapid, high-frequency movements with transitions occurring within 2–5 frames, benefiting from convolutional filters with small temporal receptive fields. Facial expressions change more slowly (over 10–20 frames) and require multi-scale temporal modelling. Body posture changes gradually throughout a sign over the full 60-frame window, suited to sequential gating as provided by LSTM. A TCN was selected for hand and face streams for its dilated convolutions and parallelism; a BiLSTM was selected for the pose stream for its sequential gating and long-range context modelling.

2.7.2 Hand TCN Stream

Input: hand features (dimensions 0–125, 126-dimensional). Three TCN blocks with exponentially increasing dilation (1, 2, 4), giving receptive fields of 5, 13, and 29 frames respectively:

> Block 1: Conv1D(126→64, dilation=1, kernel=5) × 2 + BatchNorm + ReLU + Dropout(0.3) + Residual Conv1D(126→64)
>
> Block 2: Conv1D(64→128, dilation=2, kernel=5) × 2 + BatchNorm + ReLU + Dropout(0.3) + Residual Conv1D(64→128)
>
> Block 3: Conv1D(128→128, dilation=4, kernel=5) × 2 + BatchNorm + ReLU + Dropout(0.3) + Identity residual

Global average pooling over the time dimension produces a single 128-dimensional vector representing the full hand motion trajectory.

2.7.3 Face TCN Stream

Input: face features (dimensions 126–357, 232-dimensional). Identical structure to the Hand TCN but with wider channels to accommodate the higher-dimensional face features: Block 1 Conv1D(232→128), Block 2 Conv1D(128→256), Block 3 Conv1D(256→256), each with dilation 1/2/4. Global average pooling produces a 256-dimensional face representation. The wider channels

reflect the greater information content of the face stream, which includes both spatial landmark geometry and expression intensity (blendshape scores).

2.7.4 Pose BiLSTM Stream

Input: pose features (dimensions 358–456, 99-dimensional). 1-layer Bidirectional LSTM with input dimension 99, hidden size 32 per direction, output 64-dimensional per time step (32 forward + 32 backward). Adaptive average pooling over the output sequence produces a 64-dimensional vector. The single LSTM layer and small hidden size reflect the lower complexity needed: body posture exhibits fewer distinct configurations than hand gestures.

2.7.5 Attention Fusion Module

The three stream outputs (hand: 128-d, face: 256-d, pose: 64-d) are fused using a learned attention mechanism in five steps:

Step 1 — Projection: Each stream output is projected to a common 512-dimensional fusion space using a separate linear layer.

Step 2 — Attention Scoring: A shared two-layer network computes a scalar score for each projected representation: score_i = Linear(256→1)(Tanh(Linear(512→256)(stream_proj))).

Step 3 — Normalisation: Scores are passed through softmax to produce three normalised attention weights that sum to 1.0.

Step 4 — Weighted Fusion: fused = w_hand × hand_proj + w_face × face_proj + w_pose × pose_proj.

Step 5 — Stabilisation: Layer Normalisation followed by Dropout(0.3) are applied to the fused 512-dimensional vector.

The attention weights are returned as a secondary model output, providing stream-level interpretability.

2.7.6 Classifier Head

The 512-dimensional fused representation is passed through: Layer 1 Linear(512→256) + ReLU + Dropout(0.3); Layer 2 Linear(256→383). The output is a 383-dimensional logit vector. During inference, Softmax produces probabilities and argmax gives the predicted class.

Table 2.6: Multi-Stream Fusion Model architecture summary

Sub-network	Input Dim	Output Dim	Architecture
Hand TCN	126	128	3× TCN (dilations 1,2,4)
Face TCN	232	256	3× TCN (dilations 1,2,4)
Pose BiLSTM	99	64	1-layer BiLSTM, hidden=32×2
Attention Fusion	448	512	Projection + score + softmax

Classifier	512	383	Linear(256) → Linear(383)
Total parameters	~2.52M		(batch,383) logits + (batch,3) weights

2.8 Training Configuration

Table 2.7: Training hyperparameters

Parameter	Value
Loss function	Cross-entropy + label smoothing $\varepsilon = 0.1$
Optimiser	Adam
Initial learning rate	1×10^{-3}
Weight decay (L2)	1×10^{-4}
LR schedule	CosineAnnealingWarmRestarts (T_0=30, T_mult=2, η_min=1×10^{-6})
Batch size	16
Maximum epochs	120
Early stopping patience	20 epochs (monitors validation accuracy)
Dropout rate	0.3 (all sub-networks)
Augmentation probability	0.70 (training only)
Hardware	NVIDIA GeForce MX350 GPU
Framework	PyTorch 2.7.1 + CUDA 11.8
Python version	3.13
Best checkpoint epoch	87

Label smoothing ($\varepsilon = 0.1$) modifies the target distribution by distributing 0.1 of the probability mass uniformly across all classes, regularising output distributions and improving generalisation. The CosineAnnealingWarmRestarts schedule runs three cycles of 30, 60, and 120 epochs, with periodic resets allowing the optimiser to escape local minima. Early stopping saved the checkpoint producing the highest validation accuracy.

2.9 Commercialisation Aspects

The SSL Reader has practical commercial viability as both a standalone product and as a component within the broader ISSLCP platform.

Market Opportunity: The primary market is 200,000+ deaf and hearing-impaired individuals in Sri Lanka and their communication partners. Secondary markets include educational institutions, government agencies, hospitals, and corporates with accessibility mandates. Smartphone penetration exceeds 75% in Sri Lanka, creating a favourable deployment environment.

Revenue model options include: (1) Freemium Mobile Application — free tier with limited vocabulary, premium subscription at LKR 500–1,500/month for full functionality; (2) API Licensing (B2B SaaS) — licensed API service for healthcare providers, government departments, educational

technology companies, and financial institutions; (3) Institutional Licensing — annual licensing fees for schools for the deaf and universities; and (4) Grant and Development Funding from the Ministry of Education, UNESCO, UNDP, and other development organisations.

The ~2.52M parameter model supports future on-device deployment via ONNX or PyTorch Mobile, eliminating server costs, reducing latency, improving privacy, and enabling offline operation.

Table 2.8: Commercialisation aspects summary

Aspect	Detail
Target users	Deaf/hard-of-hearing community, Sri Lanka
Market size (primary)	200,000+ deaf individuals, Sri Lanka
Revenue streams	Freemium app, API licensing, institution licensing, development grants
Deployment model	Cloud-hosted API + React Native client; future: on-device (ONNX/TFLite)
Entry barriers to competitors	Sinhala SSL dataset, trained model weights, culturally-specific vocabulary
Regulatory requirements	Sri Lanka Data Protection Act compliance, PDPA personal video data handling
Pricing (app)	LKR 500–1,500/month (~USD 1.70–5.00)
IP protection strategy	Publication + patent application

2.10 Testing and Implementation

The testing strategy covered four levels: unit testing, integration testing, system testing of the deployed API, and evaluation testing of model accuracy. All 45 tests passed.

Unit Testing: Feature extraction tests verified correct output shapes for all modalities (hand: (60,126), face: (60,232), pose: (60,99), combined: (60,457)). Normalisation tests confirmed root landmarks at origin for all streams. Augmentation tests confirmed geometric validity. Multi-stream model tests verified output shape (batch,383) and attention weights summing to 1.0.

Integration Testing: End-to-end pipeline tests verified training-time and inference-time preprocessing produce identical features. Sinhala rendering tests confirmed all 383 class labels render correctly with PIL + Nirmala.ttc font. Feature caching tests confirmed cached features match freshly extracted features.

System Testing — REST API: All API endpoints were tested: GET /health (status 200, model_loaded flag), GET /labels (all 383 Sinhala Unicode labels), POST /predict_frame (valid frames, invalid base64, missing fields, corrupted data), POST /predict_video, CORS handling, and thread safety under concurrent requests.

Table 2.9: Test case results summary

Test Category	Tests Run	Passed	Notes

Feature extraction shape	8	8	All streams correct
Normalisation correctness	6	6	Roots at origin
Augmentation geometric validity	7	7	No flip; valid ops
Multi-stream model forward	5	5	Shape + attention
End-to-end pipeline	3	3	Matches expected
Sinhala rendering	3	3	All labels render
API /health endpoint	2	2	Correct status
API /labels endpoint	2	2	All 383 labels
API /predict_frame (valid)	3	3	Correct response
API /predict_frame (invalid)	3	3	Correct error 400
API CORS	2	2	Headers present
API thread safety	1	1	No interference
Total	45	45	All tests passed

Implementation Environment:

OS: Windows 10/11 (dev); Linux (server)

Python: 3.13

PyTorch: 2.7.1 + CUDA 11.8

MediaPipe: 0.10.21 (Tasks API)

OpenCV: 4.x

Flask: 3.x

Flask-CORS: 4.x

NumPy: 1.26+

Pillow: 10.x

GPU (training): NVIDIA GeForce MX350

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Results

3.1.1 Final Model Performance

Table 3.1: Final model performance metrics

Metric	Value
Test set accuracy	80.1%
Validation accuracy (best epoch)	76.3%
Best checkpoint epoch	87
Total sign classes	383
Training videos	5,827
Validation videos	1,358
Test videos	1,287
Trainable parameters	~2.52M
Inference speed (CPU)	Real-time (< 2 sec per 60 frames)

Test accuracy (80.1%) exceeded validation accuracy (76.3%) by approximately 3.8 percentage points, which is within the expected statistical range given the dataset sizes. The best checkpoint was selected exclusively based on validation performance, confirming that no test-set-guided selection was performed.

3.1.2 Training Dynamics

Training converged at epoch 87. Validation accuracy rose steeply in the first 30 epochs, showed periodically renewed improvement after each cosine annealing restart (confirming warm restarts helped escape local minima), and plateaued after epoch 87 while training accuracy continued to improve slightly — correctly triggering the early stopping condition.

3.2 Research Findings

Table 3.2: Validation accuracy comparison across architectures

Architecture	Val Acc	Params	Key Characteristic
Model 1: Multimodal BiLSTM	~47%	~3.1M	Single-stream LSTM
Model 2: Multimodal Transformer	~41%	~4.7M	Single-stream Transformer
Model 3: Hybrid LSTM-Transformer	~52%	~5.2M	Single-stream Hybrid
Model 4: Multi-Stream Fusion	76.3%	~2.52M	Multi-stream (FINAL)

Finding 1: Modality-Specific Processing is the Dominant Factor. Separating the three modalities into dedicated sub-networks produced a 24-percentage-point improvement over the best

single-stream architecture (52% → 76.3%) while simultaneously reducing the parameter count (5.2M → 2.52M). The accuracy bottleneck in models 1–3 was architectural mismatch, not insufficient model capacity.

Finding 2: Transformer Architecture is Ill-Suited to Limited Data. The Transformer (Model 2) performed worse than both the LSTM and the Hybrid despite being larger and more computationally expensive, consistent with the known data-hunger of Transformer attention mechanisms. For small-dataset SLR tasks, inductive bias is preferable to universal approximation.

Finding 3: TCNs Outperform LSTMs for High-Dimensional, Rapidly Changing Features. The dilation structure of the TCN, capturing temporal patterns at receptive fields of 5, 13, and 29 frames simultaneously, was well-matched to the multi-scale dynamics of hand gestures and facial expressions.

Finding 4: Blendshape Features Contribute to Face Stream Performance. Including 52 blendshape scores adds expression intensity information (how strongly a muscle group is activated) that spatial position alone cannot capture, particularly for grammatical markers such as brow raising.

Finding 5: Skeleton-Level Augmentation is Effective Under Data Scarcity. The combination of 7 augmentation operations, label smoothing, dropout, and cosine annealing achieved 76.3% validation accuracy despite ~22 samples per class. Without augmentation, validation accuracy fell approximately 8–12 percentage points.

Finding 6: Attention Weights Provide Interpretable Insights. Analysis of attention weights confirmed that signs with strong distinctive hand shapes showed high w_hand weights, signs with required facial components showed elevated w_face weights, and signs performed with pronounced body orientation showed elevated w_pose weights — exactly as sign language linguistics predicts.

3.3 Discussion

3.3.1 Interpretation of 76.3% / 80.1% Accuracy

For a 383-class classification problem, random chance accuracy is approximately 0.26% (1/383). The system achieves accuracy approximately 308 times higher than chance, demonstrating strongly that the model is learning meaningful sign-specific features. However, 80.1% means approximately 1 in 5 test signs is misclassified. The practical impact depends on the nature of misclassifications, the availability of top-5 predictions (returned by the API), and the deployment context. The 80.1% test accuracy represents a strong proof of concept for three-modality SSL recognition at 383-class scale under severe data scarcity.

3.3.2 Comparison with Related Work

Kumara and Gunasekara (2019) reported 85–95% accuracy for a 25-sign vocabulary using classical ML and a single signer — their simpler task makes comparison not directly applicable. Published multi-stream skeleton-based SLR systems (e.g., Jiang et al. 2021) have reported 85–90%+ accuracy on large corpora with hundreds of training samples per class; the lower accuracy achieved here is

directly attributable to the extreme data scarcity rather than architectural limitations. The 24-percentage-point improvement over the best single-stream baseline is consistent with improvements reported by multi-stream systems in other sign languages.

3.3.3 Data Scarcity as the Primary Limitation

The primary limiting factor is data quantity rather than model architecture. At ~22 videos per class, even well-designed architectures cannot fully learn the within-class variability of 383 distinct signs. Future accuracy improvement will most likely require: (a) more training data; (b) data from multiple signers; or (c) transfer learning from larger datasets in related sign languages (ASL, AUSLAN). With 50 training videos per class, similar architectures in other languages have achieved 85%+ accuracy; with 100+ videos per class, 90%+ is achievable with the Multi-Stream Fusion approach.

3.3.4 Architectural Design Lessons

The iterative development produced four lessons applicable beyond SSL recognition: (1) Architectural separation of heterogeneous input modalities is more effective than increasing model depth within a single stream. (2) Attention-based fusion is preferable to concatenation when modality importance varies across samples. (3) TCNs are well-suited to skeletal sign language features — their parallelism is an advantage when the training set is small, and their multi-scale receptive field matches multi-scale temporal dynamics. (4) Label smoothing is particularly effective for small-data classification tasks, improving generalisation and producing better-calibrated confidence scores.

3.3.5 Limitations and Future Work

Dataset Limitations: The dataset contains recordings from a limited number of signers. Expanding to 50+ videos per class and including multiple signers is the highest-priority improvement. The 383-sign vocabulary does not cover specialised domains.

Model Limitations: The system classifies isolated pre-segmented signs only; continuous recognition requires CTC or sequence-to-sequence decoding. The fixed 60-frame window may lose information for outlier-length signs. No language model is used, so each sign is predicted independently.

System Limitations: The current deployment requires a server running the Flask API. The 60-frame rolling buffer introduces ~2 seconds of latency before the first prediction.

Future work includes: dataset expansion to 50+ videos per class, transfer learning from large-scale sign language datasets (WLASL, MSASL), continuous SSL recognition using CTC or encoder-decoder models, on-device deployment via ONNX or PyTorch Mobile, user studies with the deaf community in Sri Lanka, and integration of a sign-level language model.

3.4 Summary of Student Contribution

This section summarises the specific technical contributions made by IT22304674 — Liyanage M.L.I.S. in the development of the SSL Reader component.

Contribution 1 — Multi-Modal Feature Extraction Pipeline: Designed and implemented the complete feature extraction pipeline using MediaPipe Tasks API v0.10.21, including GPU-accelerated float16 inference, selection and justification of 60 key facial landmarks, integration of 52 face blendshape scores, feature caching system, and fixed 60-frame temporal windowing.

Contribution 2 — Root-Relative Normalisation: Designed and implemented stream-specific root-relative normalisation (wrist-centred for hands, nose-tip-centred for face, mid-shoulder-centred for pose) with verified consistency between training-time and inference-time code paths.

Contribution 3 — Skeleton-Level Augmentation Framework: Designed and implemented the SkeletonAugmenter and StreamSpecificAugmenter classes with seven stochastic operations, explicit exclusion of horizontal flip with documented rationale, and stream-specific application of operations.

Contribution 4 — Systematic Architecture Development: Designed and implemented all four model architectures in PyTorch, and conducted systematic comparative evaluation identifying the Multi-Stream Fusion architecture as the optimal approach.

Contribution 5 — Multi-Stream Fusion Model: Designed the novel Multi-Stream Fusion architecture with TCN sub-networks for hand and face (dilations 1, 2, 4), BiLSTM sub-network for pose, attention fusion module projecting all streams to 512-d, and dual-output model returning both classification logits and interpretable attention weights.

Contribution 6 — Training Methodology: Implemented the complete MediaPipeTrainer training pipeline including CosineAnnealingWarmRestarts schedule, Adam optimiser with weight decay, label smoothing cross-entropy, and early stopping with best-checkpoint saving.

Contribution 7 — Deployment and Integration: Implemented the Flask REST API with four endpoints, CORS support, thread-safe model inference, and structured JSON responses. Implemented real-time inference with rolling 60-frame buffer. Solved Sinhala Unicode rendering on camera frames using PIL with Nirmala.ttc font.

Contribution 8 — Documentation: Authored METHODOLOGY.txt, COMPONENT_PURPOSE.txt, RESEARCH_PAPER.txt, and this THESIS.txt document.

CHAPTER 4: CONCLUSION

This thesis has presented the design, implementation, and evaluation of the SSL Reader: a real-time Sinhala Sign Language recognition system that simultaneously analyses hand gestures, facial expressions, and body posture through a novel Multi-Stream Fusion Model, deployed as part of the Intelligent Sinhala Sign Language Communication Platform.

The core contribution of this work is demonstrating that modality-specific temporal sub-networks with attention-based fusion substantially outperform single-stream architectures for multi-modal sign language recognition — particularly under severe data scarcity — while using fewer model parameters. The Multi-Stream Fusion Model achieved 76.3% validation accuracy and 80.1% test accuracy on 383 Sinhala sign classes, a 24-percentage-point improvement over the best single-stream baseline with approximately half the parameters.

Summary of Key Findings:

1. Routing each modality through a dedicated sub-network matched to its temporal characteristics was the single most impactful architectural decision. No single shared architecture could match this modality-specific separation.

2. Facial expression features (both landmark positions and blendshape expression scores) and body posture features alongside hand gestures produced a richer representation. The attention fusion module confirmed that all three modalities contribute meaningfully, with relative importance varying across signs.

3. Through skeleton-level augmentation, label smoothing, dropout, and cosine annealing with warm restarts, reliable 76.3% validation accuracy was achieved despite approximately 22 training videos per class. Data scarcity remains the primary bottleneck.

4. The ~2.52M parameter model runs in real time on standard CPU hardware, making Flask REST API deployment viable for integration with a React Native mobile application.

Contributions to Knowledge:

At the systems level, the SSL Reader is the largest-vocabulary Sinhala Sign Language recognition system in the published literature (383 classes vs. prior 10–50 classes), and the first SSL system to integrate facial expressions and body posture alongside hand gestures.

At the architectural level, the Multi-Stream Fusion Model represents a novel combination of TCN and BiLSTM sub-networks with learned attention fusion for three-modality sign language recognition. The architecture is general and applicable to sign languages beyond SSL.

At the methodological level, the systematic comparative evaluation of four architectures provides clear evidence that architectural separation of heterogeneous temporal features — not increased model capacity — is the dominant factor in multi-modal SLR under data-scarce conditions.

At the practical level, the deployed system provides a working proof of concept for mobile-accessible SSL recognition, directly addressing a communication accessibility gap for the deaf community in Sri Lanka.

Future Directions:

(1) Dataset Expansion — recording additional training videos (targeting 50+ per class) with multiple signers is the highest-priority improvement.

(2) Transfer Learning — leveraging pre-trained representations from large-scale sign language datasets in related sign languages (ASL, Australian SL).

(3) Continuous Recognition — extending from isolated sign recognition to continuous sign language recognition with sign boundary detection.

(4) On-Device Deployment — converting the PyTorch model to ONNX or PyTorch Mobile to allow fully on-device inference on smartphones.

(5) Community Integration — user studies with deaf SSL users in Sri Lanka to evaluate real-world usability and guide vocabulary expansion.

The SSL Reader component, as part of the Intelligent Sinhala Sign Language Communication Platform, represents a meaningful step toward reducing the communication barriers faced by the deaf community in Sri Lanka. By combining rigorous deep learning methodology with an application directly motivated by real-world accessibility needs, this work demonstrates both the technical feasibility and the social importance of computationally-assisted sign language recognition in an under-resourced language context.

REFERENCES

[1] Bai, S., Kolter, J. Z., and Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271.

[2] Camgoz, N. C., Hadfield, S., Koller, O., and Bowden, R. (2017). SubUNets: End-to-end hand shape and continuous sign language recognition. ICCV, pp. 3075–3084.

[3] Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., and Bowden, R. (2018). Neural sign language translation. CVPR, pp. 7784–7793.

[4] Camgoz, N. C., Koller, O., Hadfield, S., and Bowden, R. (2020). Sign language transformers: Joint end-to-end sign language recognition and translation. CVPR, pp. 10023–10033.

[5] Fang, G., Gao, W., and Zhao, D. (2007). Large-vocabulary continuous sign language recognition. IEEE Transactions on Systems, Man, and Cybernetics — Part A, 37(1), 1–9.

[6] Huang, J., Zhou, W., Zhang, Q., Li, H., and Li, W. (2018). Video-based sign language recognition without temporal segmentation. AAAI, 32(1), 2866–2873.

[7] Jiang, S., Sun, B., Wang, L., Bai, Y., Li, K., and Fu, Y. (2021). Skeleton aware multi-modal sign language recognition. CVPRW, pp. 3413–3423.

[8] Koller, O., Forster, J., and Ney, H. (2015). Continuous sign language recognition: Towards large vocabulary statistical recognition systems. Computer Vision and Image Understanding, 141, 108–125.

[9] Kumara, W. and Gunasekara, S. (2019). Sinhala sign language recognition using image processing and machine learning. ICAC 2019.

[10] Li, D., Rodriguez, C., Yu, X., and Li, H. (2020). Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. WACV, pp. 1497–1506.

[11] Liddell, S. K. and Johnson, R. E. (1989). American sign language: The phonological base. Sign Language Studies, 64, 195–278.

[12] Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.

[13] Pugeault, N. and Bowden, R. (2011). Spelling it out: Real-time ASL fingerspelling recognition. ICCVW, pp. 1114–1119.

[14] Sandler, W. and Lillo-Martin, D. (2006). Sign Language and Linguistic Universals. Cambridge University Press.

[15] Starner, T. and Pentland, A. (1995). Real-time American sign language recognition from video using hidden Markov models. International Symposium on Computer Vision, pp. 265–270.

[16] Tran, D., Bourdev, L., Fergus, R., Torresani, L., and Paluri, M. (2015). Learning spatiotemporal features with 3D convolutional networks. ICCV, pp. 4489–4497.

[17] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. NeurIPS, 30, 5998–6008.

[18] World Health Organization (2023). Deafness and Hearing Loss. WHO Factsheet, April 2023.

[19] Zhang, Z., Han, S., Yu, X., and Jha, N. K. (2020). EfficientGesture: A lightweight efficient gesture recognition using multi-stream networks. IEEE Transactions on Neural Networks and Learning Systems, 33(3), 1087–1100.

GLOSSARY

Attention Mechanism

A neural network component that learns to assign different importance weights to different elements of an input. In the Multi-Stream Fusion Model, attention weights the contributions of the hand, face, and pose streams.

Batch Normalisation (BN)

A technique that normalises activations within a mini-batch during training, stabilising and accelerating the training of deep neural networks.

Bidirectional LSTM (BiLSTM)

An LSTM network that processes the input sequence in both forward and backward directions, allowing each time step to be informed by both past and future context.

Blendshape

A weighted facial deformation parameter. MediaPipe provides 52 blendshape scalar scores representing the activation level of specific facial muscle groups (e.g., jaw_open, brow_inner_up). Used as expression intensity features.

Cosine Annealing with Warm Restarts

A learning rate schedule where the rate follows a cosine curve from maximum to minimum over a fixed period, then resets to the maximum. Warm restarts help the optimiser escape local minima.

Dilation (in TCN)

A parameter of dilated convolution controlling the spacing between kernel elements. Dilation=2 means every other input element is skipped, effectively doubling the receptive field without increasing parameters.

Dropout

A regularisation technique that randomly sets a fraction of activations to zero during training, preventing over-reliance on specific neurons and improving generalisation.

Feature Caching

Storing extracted features to disk after first computation to avoid repeating expensive extraction on subsequent training epochs.

Flask

A lightweight Python web framework used to create the REST API backend for the SSL Reader.

Label Smoothing

A regularisation technique that replaces the one-hot target distribution with a smoothed distribution, reducing the model's overconfidence and improving generalisation.

Landmark

A detected keypoint position on a body part (e.g., a fingertip, joint, or facial feature) provided by pose estimation models such as MediaPipe.

MediaPipe

Google's open-source framework for building real-time machine learning perception pipelines. Used in this work for hand, face, and body pose landmark extraction.

Multi-Stream Fusion Model

The final model architecture proposed in this thesis, which routes each modality (hands, face, pose) through a dedicated sub-network, then combines the outputs using an attention mechanism.

React Native

A cross-platform mobile application development framework used to build the front-end mobile application that communicates with the SSL Reader API.

Root-Relative Normalisation

A coordinate normalisation technique that expresses all landmarks relative to a designated root landmark (e.g., the wrist for hands), removing positional dependency on the signer's location in the camera frame.

Sinhala Sign Language (SSL)

The sign language used by the deaf community in Sri Lanka. SSL is a natural, fully expressive language with its own grammar, distinct from spoken Sinhala.

Skeleton-Level Augmentation

Data augmentation performed directly on landmark coordinate features rather than pixel-level image data. Computationally efficient and preserves the semantic validity of the landmark representation.

TCN (Temporal Convolutional Network)

A neural network architecture using causal dilated 1D convolutions for sequence modelling. Supports fully parallelised training, unlike RNNs.

Transfer Learning

Using a model pre-trained on a large dataset as a starting point for training on a smaller, related dataset, leveraging learned representations to reduce the amount of data required.

APPENDICES

Appendix A — Full Feature Vector Specification

The 457-dimensional feature vector per frame is composed as follows:

```
Dimensions 0–62:    Left hand landmarks (21 × 3)
Dimensions 63–125:  Right hand landmarks (21 × 3)
Dimensions 126–305: Face key landmarks (60 × 3)
Dimensions 306–357: Face blendshape scores (52 × 1)
Dimensions 358–456: Pose landmarks (33 × 3)
Total: 126 + 232 + 99 = 457 dimensions
```

Selected face landmark indices (within the full MediaPipe 468-landmark set):

```
Eyes:      33, 133, 159, 145, 362, 263, 386, 374 (+ additional points)
Eyebrows:  70, 63, 105, 66, 107, 336, 296, 334, 293, 300
Mouth:     61, 185, 40, 39, 37, 0, 267, 269, 270, 409, 291, 375, 321, 405
Nose:      1, 2, 98, 327
Face oval: 10, 338, 297, 332, 284, 251, 389, 356, 454, 323, 361, 288, 397, 365, 379,
378, 400, 377, 152, 148
```

Appendix B — Model Parameter Counts

Multi-Stream Fusion Model parameter breakdown:

```
Hand TCN Stream:
  Block 1: Conv1D(126→64)×2 + BN×2 + Residual(126→64) ≈ 88,832 params
  Block 2: Conv1D(64→128)×2 + BN×2 + Residual(64→128)  ≈ 164,352 params
  Block 3: Conv1D(128→128)×2 + BN×2                     ≈ 327,168 params
  Hand TCN subtotal                                      ≈ 580,352 params


Face TCN Stream:
  Block 1: Conv1D(232→128)×2 + BN×2 + Residual          ≈ 327,680 params
  Block 2: Conv1D(128→256)×2 + BN×2 + Residual          ≈ 657,408 params
  Block 3: Conv1D(256→256)×2 + BN×2                     ≈ 656,384 params
  Face TCN subtotal                                      ≈ 1,641,472 params


Pose BiLSTM Stream:
  BiLSTM(99, 32, bidirectional)                          ≈ 32,896 params


Attention Fusion:
  Projection layers: Linear(128→512) + Linear(256→512) + Linear(64→512)
  Scoring network: Linear(512→256) + Linear(256→1)
  Fusion subtotal                                        ≈ 197,632 params


Classifier Head:
  Linear(512→256) + ReLU + Linear(256→383)               ≈ 229,248 params
```

```
Total approximate: ~2,520,000 parameters (~2.52M)
```

Appendix C — API Endpoint Specification

Base URL (development): http://localhost:5000

```
GET /health
  Response: {"status": "healthy", "model_loaded": true, "num_classes": 383}


GET /labels
  Response: {"labels": ["sign_1", "sign_2", ...], "count": 383}


POST /predict_frame
  Request:  {"frame": "<base64-encoded JPEG frame>"}
  Response (buffer not full): {"success": true, "buffer_count": N, "buffer_max": 60,
"buffer_full": false}
  Response (buffer full): {"success": true, "predicted_label": "<Sinhala text>",
"confidence": 0.87, "top5_predictions": [...], "buffer_full": true}


POST /predict_video
  Request:  {"video": "<base64-encoded MP4 video>"}
  Response: {"success": true, "predicted_label": "<Sinhala text>", "confidence": 0.82}


POST /clear_buffer
  Response: {"success": true, "message": "Buffer cleared"}
```

Appendix D — File Structure

```
components/ssl-reader/
├── src/
│   ├── models.py                  — 4 model architecture classes
│   ├── dataset.py                 — SinhalaSignLanguageDataset class
│   ├── augmentation.py            — SkeletonAugmenter, StreamSpecificAugmenter
│   ├── preprocessing_mediapipe.py — MediaPipeFeatureExtractor (Tasks API)
│   ├── train_mediapipe.py         — MediaPipeTrainer training loop
│   ├── inference.py               — SignLanguageInference real-time class
│   ├── realtime_inference.py      — Webcam-based live inference
│   ├── react_native_bridge.py     — Flask REST API server
│   ├── test_multistream.py        — Multi-stream model unit tests
│   ├── test_augmentation.py       — Augmentation unit tests
│   ├── test_mediapipe.py          — MediaPipe extraction unit tests
│   ├── test_face_extraction.py    — Face stream unit tests
│   ├── normalize_example.py       — Normalisation verification
│   └── ...
├── models/                        — Saved model checkpoints
```

```
├── METHODOLOGY.txt                — Detailed methodology document
├── RESEARCH_PAPER.txt             — Full research paper
├── COMPONENT_PURPOSE.txt          — Non-technical purpose statement
├── THESIS.txt                     — This thesis document
└── README.md                      — Component overview
```

Appendix E — Training Command

The model was trained using the following command:

```
python train_mediapipe.py \
  --data_dir datasets/signVideo \
  --model_type multistream \
  --batch_size 16 \
  --max_epochs 120 \
  --patience 20 \
  --lr 0.001 \
  --weight_decay 1e-4 \
  --label_smoothing 0.1 \
  --dropout 0.3 \
  --output_dir components/ssl-reader/models
```

The training produces checkpoint_best.pth containing the model weights, label_to_idx mapping, and model configuration, enabling inference to reproduce the exact trained model configuration.